FH-OÖ Hagenberg/HSD
Embedded Visualization with Qt
Übung 1 Simplechat Client (_____ / 100 Punkte)

Ziel: Umsetzung eines SimpleChat Clients für den RaspberryPI mit QWidgets, erweitert um eigene Design-Ideen

Als Basis dient das Qt-Beispiel „Simple WebSocket Chat“ (QtCreator – Beispiele – „simplechat“). Es enthält den SimpleChatServer (Konsolenanwendung mit QWebSocketServer) und einen HTML5-Client (chatclient.html). Folgender Screenshot des HTML5-Clients soll sinngemäß auch am RaspberryPI funktionieren:



Teil 1 – Basis-Client

Erstelle einen lauffähigen Prototyp, der optimiert auf die Platzverhältnisse des Raspberry Pi Bildschirmes ist (800x480):

- Die Elemente im Screenshot oben sind nicht optimal angeordnet. Versuche mit QLayouts eine gute Anordnung der Elemente zu erstellen und somit auch eine flexible Darstellung der Elemente zu erreichen.
- Die Benutzeroberfläche soll nicht mit dem QtDesigner erstellt werden. Layouting und Strukturierung der GUI-Komponenten sollen im C++ Code vorgenommen werden.
- Skizziere vor der Umsetzung das Layout für den 800x480-Bildschirm als Handskizze. Zeichne darin die verwendeten Komponenten (Widgets) und ihre Anordnung in QLayouts (z.B. QVBoxLayout, QHBoxLayout, QGridLayout) ein. Die Skizze ist Teil der Lösungsidee.
- Da der Chat-Client als Embedded-Anwendung ausgerichtet sein soll, ist es nicht notwendig, Fensterrahmen oder Menüleisten anzuzeigen. Der Inhalt des Bildschirmes soll vollflächig mit den dargestellten Inhalten genutzt werden. (Tipp: Methode showFullScreen() nutzen)
- Nutze setStyleSheet(), um auf den Buttons zu visualisieren, ob eine aktive Verbindung (=grün) besteht oder keine Verbindung (=Default-Farbe des Buttons) besteht.

- Nutze Stylesheets oder QFont, um die Schriftgrößen auf den Buttons, Labels und Eingabefeldern zu setzen. Wichtig ist eine gute Lesbarkeit am RaspberryPI.
- Es ist nicht notwendig, eine virtuelle Software-Tastatur via Touchscreen zu implementieren. Beim Test am Raspberry PI kann eine USB-Tastatur an den Raspberry PI angesteckt werden, um Texte im QLineEdit der SimpleChat-Anwendung einzugeben.
- Unterteile die Anwendung in sinnvolle Klassen: Die GUI sollte in einer separaten Klasse abgedeckt werden. Die ChatClient-Komponente soll in einer eigenen Klasse abgedeckt werden.

Teil 2 – Erweiterung mit eigenen Design-Ideen

Erweitere den Client um Ideen, die den Chat übersichtlicher, robuster oder angenehmer bedienbar machen. Es gibt dafür keine strikten Vorgaben – entscheide selbst, was sich für einen Chat am RaspberryPI gut eignet.

- Setze **mindestens 2 Ideen aus 2 verschiedenen Kategorien** des Ideenkatalogs um. Der Ideenkatalog wird am Ende der Lehrveranstaltung vorgestellt.
- Beschreibe in der Lösungsidee, welche Ideen du gewählt hast und warum sie sich für einen Embedded-Chat eignen.

Bewertung

Bereich	**Punkte**
Teil 1: Basis-Client (Layout, Klassen, Stylesheets, Lauf am Raspberry Pi)	40
Teil 2: zwei Ideen aus dem Ideenkatalog	40
Abgabe-PDF (Lösungsidee mit Handskizze, Screenshots, Begründung der Ideen)	20
Gesamt	**100**

Hinweise für die Abgabe

Im abgegebenen gezippten File soll folgender Inhalt enthalten sein:

- Qt Namenskonventionen und Coding Style beachten
- Source-Code-Files inklusive Qt-Project File und PDF als ZIP
- Das PDF soll so aufgebaut sein:
    - Übungsangabe als Deckblatt
    - Ausgearbeitete Lösungsidee mit Handskizze des Layouts, Screenshot von der ausgeführten Raspberry-Pi-Software.
        - Hinweis: Screenshots am Raspberry-Pi können mit dem Smartphone gemacht werden.
    - Beschreibung und Begründung der umgesetzten Ideen aus Teil 2, jeweils mit einem Screenshot, der die Idee zeigt.
    - Sämtlicher Source Code in Text-Form: Qt-Project-File, main.cpp, …
- Füge der Lösungsidee auch einen Screenshot vom Desktop (Windows oder Linux, z.B. WSL) hinzu, der demonstriert, wie die Anwendung bei unterschiedlichen Fenstergrößen aussieht.
- Bereite dich darauf vor, dass du stichprobenartig vom Tutor zu deiner Lösungsidee und der Umsetzung in einem persönlichen Gespräch befragt wirst. Kannst du Teile deiner Lösung nicht erklären, werden die Punkte für diese Teile abgezogen.

Übung 1: Simplechat Client

Embedded Visualization with Qt

Simon Offenberger | S2410306027
HSD
14. November 2026

Inhaltsverzeichnis

1. Aufgabenstellung und Ziel **2**

2. Lösungsidee und Aufbau **2**
2.1. Layout und Handskizze 2
2.2. Klassen und Kommunikation 3
2.3. Darstellung und Bedienung 3

3. Erweiterungen aus Teil 2 **4**
3.1. Idee 1: Automatische Wiederverbindung 4
3.2. Idee 2: Emoji-Ersetzung 5

4. Ausführung und Nachweise **6**
4.1. Ausführung auf dem Raspberry Pi 6
4.2. Flexible Fenstergrößen am Desktop 8

A. Vollständiger Projektquellcode **10**
A.1. Projektkonfiguration 10
A.2. Programmeinstieg 13
A.3. WebSocket-Client 13
A.4. Benutzeroberfläche 17
A.5. Emoji-Ersetzung 22

1. Aufgabenstellung und Ziel

Ziel ist ein Simplechat-Client mit Qt Widgets für einen Raspberry Pi mit einer Bildschirmauflösung von 800×480 Pixeln. Die Oberfläche soll durch im C++-Code angelegte Layouts flexibel auf unterschiedliche Fenstergrößen reagieren. Die Kommunikation erfolgt über WebSockets mit dem Simplechat-Server. Zusätzlich werden zwei eigene Ideen umgesetzt.

2. Lösungsidee und Aufbau

2.1. Layout und Handskizze

Das zentrale Widget verwendet ein `QVBoxLayout`. Oben befinden sich zwei horizontale Zeilen: Serveradresse mit den Schaltflächen zum Verbinden und Trennen sowie die Eingabe des Benutzernamens. Darunter liegt der schreibgeschützte Chatverlauf als `QPlainTextEdit`. Eine weitere Zeile enthält das Nachrichteneingabefeld und die Senden-Schaltfläche.

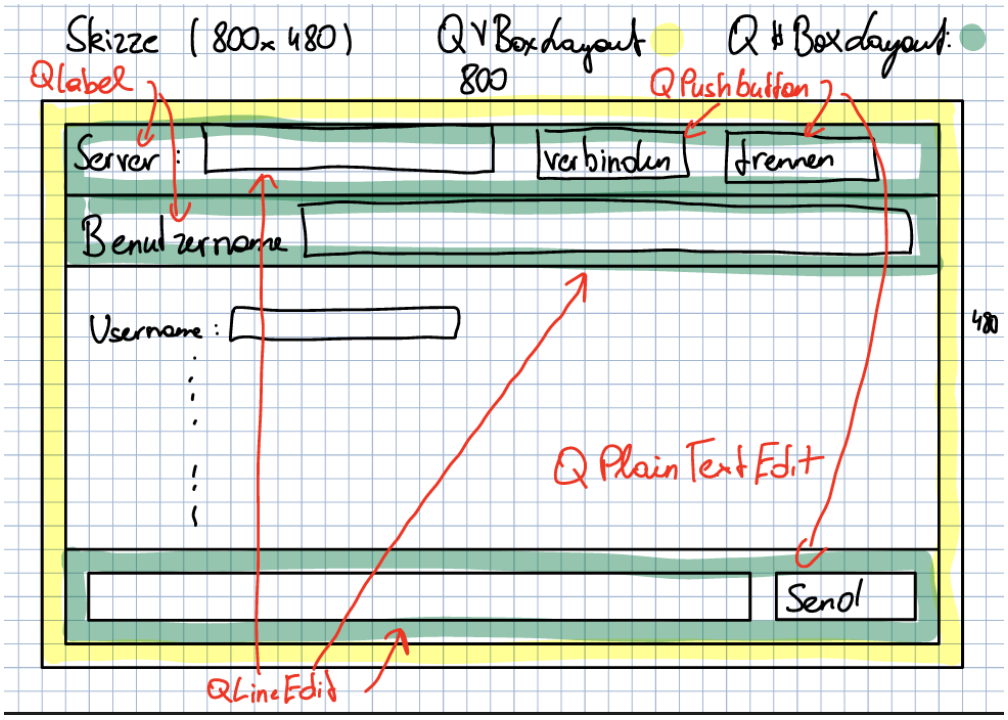


Abbildung 1: Handskizze der Oberfläche mit Widgets und QLayouts.

Für das Layout in der Skizze wurde als zentrales Layoutelement ein `QVBoxLayout` gewählt. In diesem vertikalen Layout befinden sich drei horizontale Layouts, die jeweils die Widgets der Zeilen enthalten. In der ersten Zeile befinden sich die Widgets für die Serveradresse und die Buttons zum Verbinden und Trennen. Wichtig war mir persönlich, dass hier am unteren Rand die Zeile mit der Nachrichteneingabe und der Senden-Schaltfläche ist. Denn dieses Layout ist für die meisten Menschen vertraut, da es auch in den meisten Chatprogrammen so umgesetzt ist.

2.2. Klassen und Kommunikation

`MainWindow` Erstellt die Chat-Oberfläche im C++-Code, verbindet die Bedienelemente mit Slots und stellt den Verbindungszustand dar.

`Chatclient` Kapselt den WebSocket, sendet und empfängt Nachrichten und steuert die automatische Wiederverbindung.

`EmojiReplacer` Ersetzt definierte Text-Emoticons durch Unicode-Emojis. Die Ersetzung erfolgt beim Senden und Empfangen.

`main.cpp` Erstellt die Anwendung, setzt die Schrift per Stylesheet auf 16 pt und öffnet das Hauptfenster mit `showFullScreen()`.

Signale und Slots verbinden Netzwerklogik und Oberfläche. Der Client meldet empfangene Nachrichten und Zustandsänderungen an das Hauptfenster. Beim Senden wird der Benutzername vor die Nachricht gesetzt, die Nachricht wird außerdem lokal an den Verlauf angehängt.

2.3. Darstellung und Bedienung

Bei aktiver Verbindung erscheint die Verbinden-Schaltfläche grün und zeigt „verbunden“. Im getrennten Zustand wird ihr Stylesheet zurückgesetzt. Während der Wiederverbindung ist die Anzeige gelb. Die Nachrichteneingabe und das Senden sind nur bei aktiver Verbindung freigeschaltet. Nachrichten können auch mit der Eingabetaste gesendet werden.

3. Erweiterungen aus Teil 2

Es wurden folgende zwei Ideen aus dem Ideenkatalog umgesetzt.

- Automatische Wiederverbindung bei Verbindungsabbrüchen
- Emoji-Ersetzung für die Eingaben `:)`, `:(`, `;)` und `:D`

3.1. Idee 1: Automatische Wiederverbindung

Kategorie laut Ideenkatalog: Robustheit

Umsetzung: Bei Verbindungsfehlern oder Verbindungsabbrüchen plant `Chatclient` über einen einmalig auslösenden `QTimer` einen neuen Verbindungsversuch nach 500 ms. Die zuletzt verwendete URL bleibt gespeichert. Weitere Fehlschläge können erneut einen Versuch auslösen. Manuelles Trennen stoppt den Timer und deaktiviert weitere Versuche.

Begründung: Eine vorübergehende Netzwerkunterbrechung erfordert dadurch keinen erneuten manuellen Verbindungsaufbau. Die gelbe Zustandsanzeige macht den Wiederverbindungsversuch auf dem Embedded-Gerät sichtbar.



Abbildung 2: Desktop: Wiederverbindungsversuch nach dem Stoppen des lokalen Servers.

3.2. Idee 2: Emoji-Ersetzung

Kategorie laut Ideenkatalog: Darstellung

Umsetzung: `EmojiReplacer` ersetzt die Eingaben `:)`, `:(`, `;)` und `:D` anhand einer `QMap`. Die Ersetzung erfolgt sowohl vor dem Senden als auch beim Anzeigen empfangener Nachrichten.

Begründung: Häufig verwendete Emojis lassen sich über kurze Tastatureingaben erzeugen. Eine zusätzliche Auswahloberfläche beansprucht dadurch keinen Platz auf dem kleinen Bildschirm.

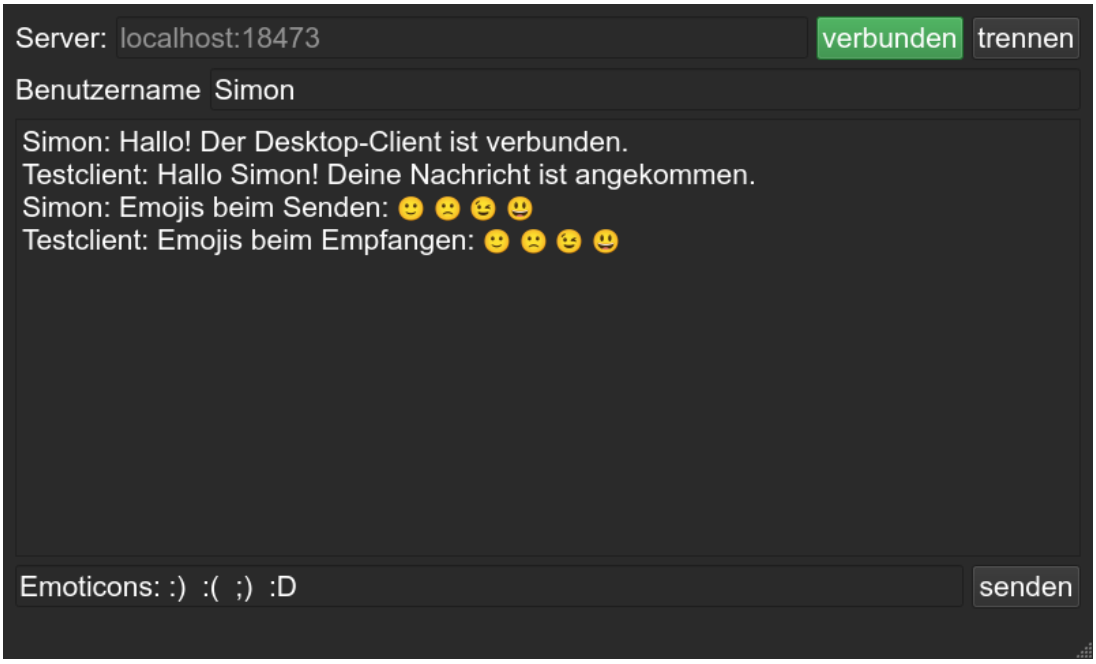


Abbildung 3: Desktop: Alle vier Emoticons werden beim Senden und Empfangen ersetzt. Die Eingabezeile zeigt die ursprünglichen Kurzformen.

4. Ausführung und Nachweise

4.1. Ausführung auf dem Raspberry Pi

Nun wurde der Chat-Client auf dem Raspberry Pi ausgeführt. Hier ist zu erkennen, dass die Oberfläche auf dem kleinen Bildschirm gut lesbar ist und die Bedienung mit den Fingern gut funktioniert. Hierbei ist es wichtig, dass die Buttons und Eingabefelder groß genug sind, um sie mit den Fingern gut bedienen zu können. Außerdem soll das Textfeld für den Chatverlauf groß genug sein, um die Nachrichten gut lesen zu können.

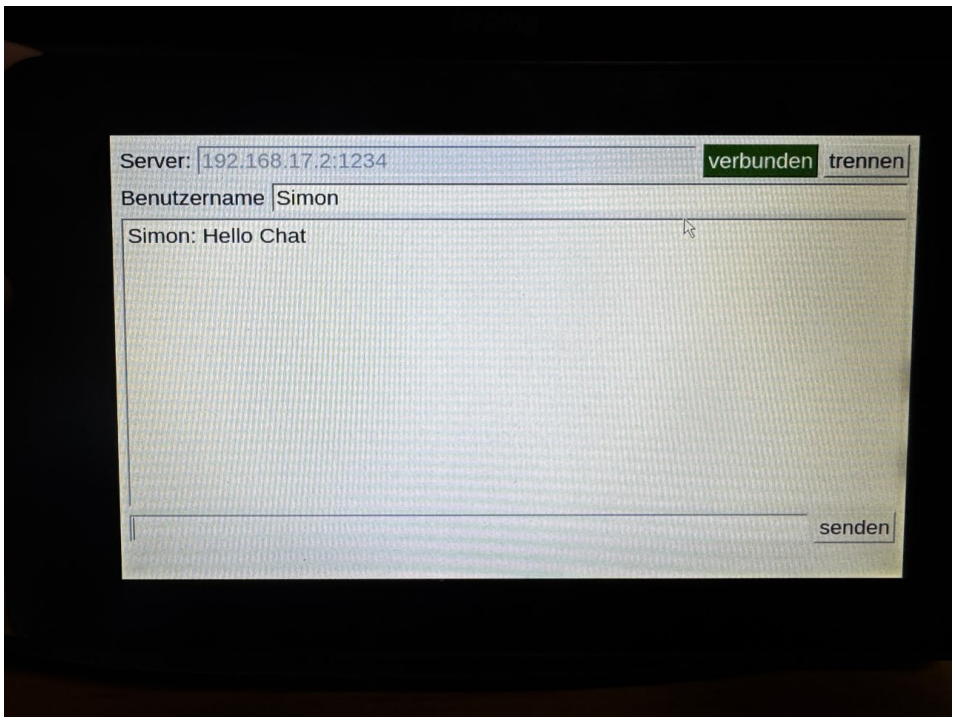


Abbildung 4: Ausgeführter Chat-Client auf dem Raspberry Pi.

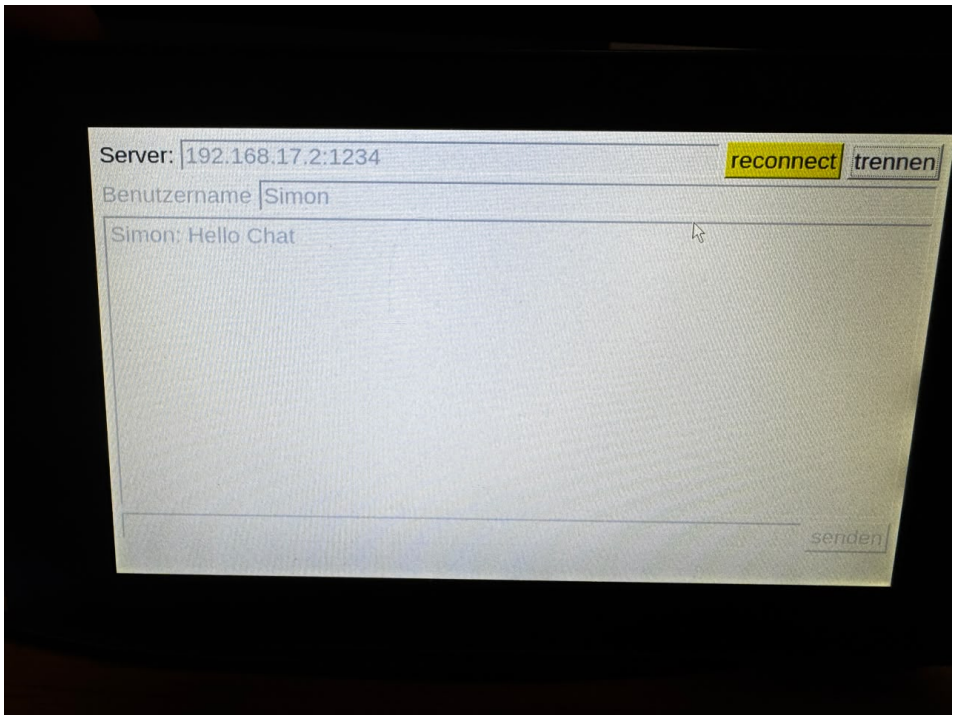


Abbildung 5: Reconnect - Wiederverbindungsversuch am Raspberry Pi.

Für die Emoji-Ersetzung auf dem Raspberry Pi musste eine Schriftart mit Emoji-Unterstützung installiert werden. Die Schriftart `Noto Color Emoji` wurde über scp auf den Raspberry Pi kopiert und anschließend eingebunden.

Hier wurde die Schriftart mit folgenden Befehlen installiert:

```
mkdir -p /usr/share/fonts/truetype/noto

cp /tmp/NotoColorEmoji.ttf \
   /usr/share/fonts/truetype/noto/

chmod 644 /usr/share/fonts/truetype/noto/NotoColorEmoji.ttf
```

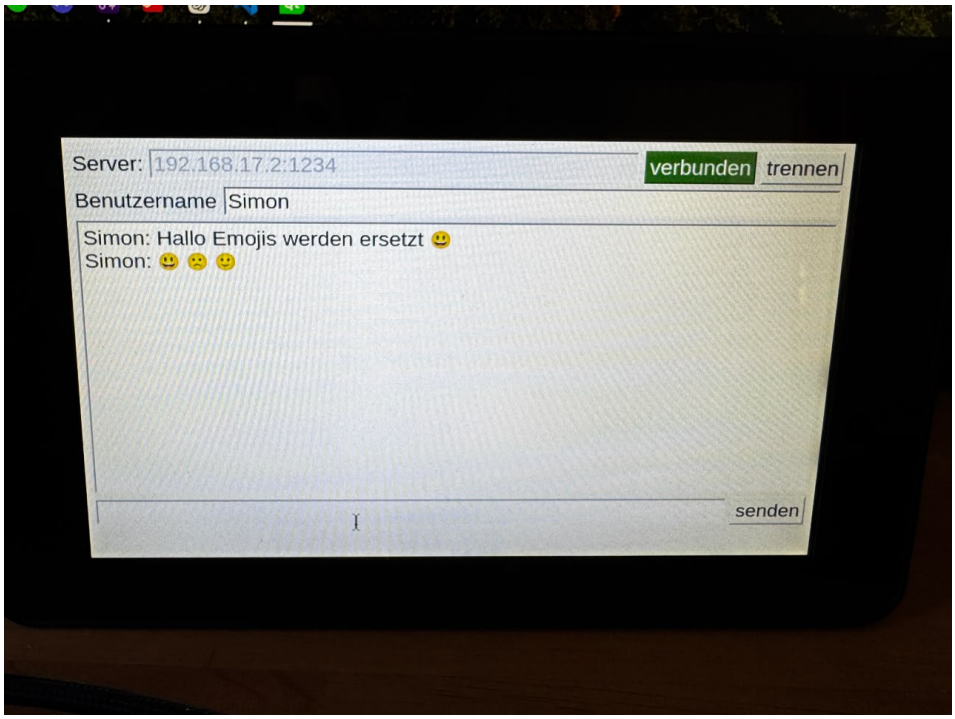


Abbildung 6: Emoji-Ersetzung auf dem Raspberry Pi.

4.2. Flexible Fenstergrößen am Desktop

Für die Desktop-Aufnahmen wurde das laufende Fenster mittels mit `show()` in den Fenstermodus versetzt und auf 800 x 480 beziehungsweise ca. 1200 x 720 Pixel skaliert. Bei größerem Fenster wachsen insbesondere der Chatverlauf und die Eingabefelder, die Anordnung der Bedienelemente bleibt erhalten. Ein zweiter WebSocket-Testclient bestätigte den Nachrichtenaustausch.



Abbildung 7: Desktop-Ausführung mit 800 x 480 Pixeln Fensterinhalt.

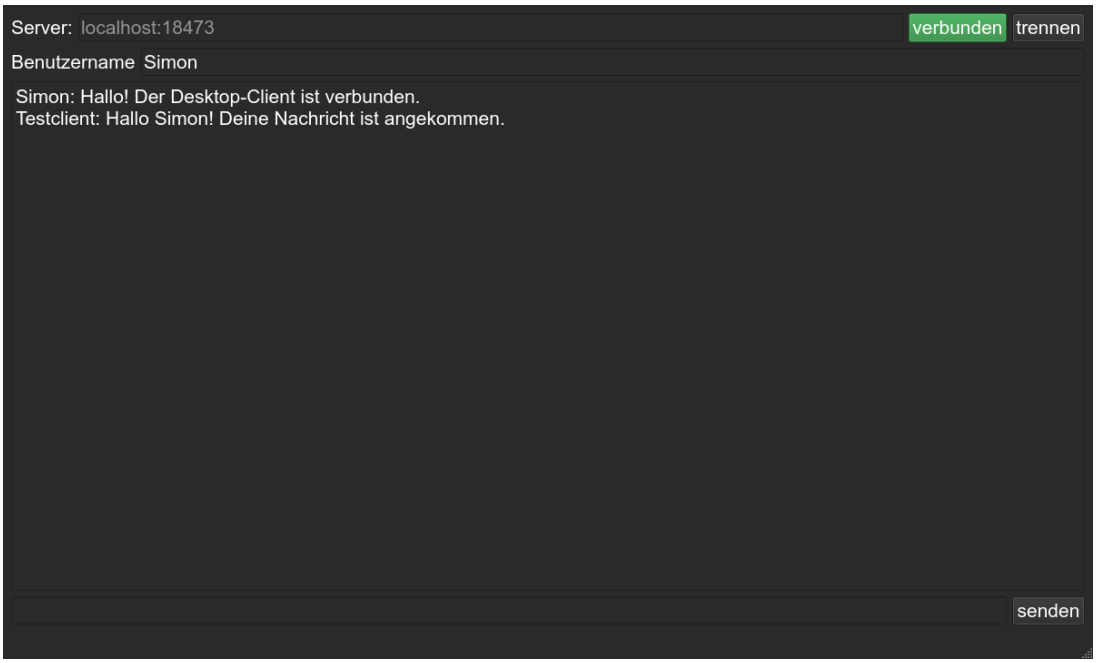


Abbildung 8: Desktop-Ausführung mit 1200 x 720 Pixeln Fensterinhalt.

A. Vollständiger Projektquellcode

Quellcodeverzeichnis

1. CMakeLists.txt . . . 10
2. CMakePresets.json . . . 11
3. main.cpp . . . 13
4. chatclient.h . . . 13
5. chatclient.cpp . . . 14
6. mainwindow.h . . . 17
7. mainwindow.cpp . . . 18
8. mainwindow.ui . . . 21
9. emojireplacer.h . . . 22
10. emojireplacer.cpp . . . 22

A.1. Projektkonfiguration

Quellcode 1: CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)

project(RPI-ChatClient VERSION 0.1 LANGUAGES CXX)

set(CMAKE_AUTOUIC ON)
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(QT NAMES Qt6 Qt5 REQUIRED COMPONENTS Widgets WebSockets)
find_package(Qt${QT_VERSION_MAJOR} REQUIRED COMPONENTS Widgets WebSockets)

set(PROJECT_SOURCES
        main.cpp
        mainwindow.cpp
        mainwindow.h
        mainwindow.ui
        chatclient.h
        chatclient.cpp
)

if(${QT_VERSION_MAJOR} GREATER_EQUAL 6)
    qt_add_executable(RPI-ChatClient
        MANUAL_FINALIZATION
        ${PROJECT_SOURCES}
        emojireplacer.h emojireplacer.cpp
    )
# Define target properties for Android with Qt 6 as:
#    set_property(TARGET RPI-ChatClient APPEND PROPERTY QT_ANDROID_PACKAGE_SOURCE_DIR
#                 ${CMAKE_CURRENT_SOURCE_DIR}/android)
# For more information, see https://doc.qt.io/qt-6/qt-add-executable.html#target-creation
else()
    if(ANDROID)
        add_library(RPI-ChatClient SHARED
            ${PROJECT_SOURCES}
        )
# Define properties for Android with Qt 5 after find_package() calls as:
```

```
#    set(ANDROID_PACKAGE_SOURCE_DIR "${CMAKE_CURRENT_SOURCE_DIR}/android")
    else()
        add_executable(RPI-ChatClient
            ${PROJECT_SOURCES}
        )
    endif()
endif()

target_link_libraries(RPI-ChatClient PRIVATE
    Qt${QT_VERSION_MAJOR}::Widgets
    Qt${QT_VERSION_MAJOR}::WebSockets
)

# Qt for iOS sets MACOSX_BUNDLE_GUI_IDENTIFIER automatically since Qt 6.1.
# If you are developing for iOS or macOS you should consider setting an
# explicit, fixed bundle identifier manually though.
if(${QT_VERSION} VERSION_LESS 6.1.0)
  set(BUNDLE_ID_OPTION MACOSX_BUNDLE_GUI_IDENTIFIER com.example.RPI-ChatClient)
endif()
set_target_properties(RPI-ChatClient PROPERTIES
    ${BUNDLE_ID_OPTION}
    MACOSX_BUNDLE_BUNDLE_VERSION ${PROJECT_VERSION}
    MACOSX_BUNDLE_SHORT_VERSION_STRING ${PROJECT_VERSION_MAJOR}.${PROJECT_VERSION_MINOR}
    MACOSX_BUNDLE TRUE
    WIN32_EXECUTABLE TRUE
)

include(GNUInstallDirs)
install(TARGETS RPI-ChatClient
    BUNDLE DESTINATION .
    LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
    RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
)

if(QT_VERSION_MAJOR EQUAL 6)
    qt_finalize_executable(RPI-ChatClient)
endif()
```

Quellcode 2: CMakePresets.json

```
{
  "version": 3,
  "cmakeMinimumRequired": { "major": 3, "minor": 21, "patch": 0 },
  "configurePresets": [
    {
      "name": "vscode",
      "displayName": "VS Code / Qt 6.8.3 Desktop",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/vscode",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_CXX_COMPILER": "/usr/bin/g++",
        "CMAKE_PREFIX_PATH": "$env{HOME}/Qt/6.8.3/gcc_64",
        "CMAKE_EXPORT_COMPILE_COMMANDS": true
      }
    }
  ],
  "buildPresets": [
    { "name": "vscode", "configurePreset": "vscode" },
    {
      "name": "vscode-autogen",
      "configurePreset": "vscode",
      "targets": ["RPI-ChatClient_autogen"]
    }
```

```
    ]
}
```

A.2. Programmeinstieg

Quellcode 3: main.cpp

```cpp
#include "mainwindow.h"

#include <QApplication>

/** @brief Starts the Qt application and shows the chat window in full screen. */
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    a.setStyleSheet(R"(
        QWidget {
            font-family: "Liberation Sans";
            font-size: 16pt;
        }
    )");

    MainWindow w;

    w.showFullScreen();
    // w.show();
    return QApplication::exec();
}
```

A.3. WebSocket-Client

Quellcode 4: chatclient.h

```cpp
#ifndef CHATCLIENT_H
#define CHATCLIENT_H

#include <QObject>
#include <QWebSocket>
#include <QTimer>

/** @brief WebSocket chat client with automatic reconnection. */
class Chatclient : public QObject
{
    Q_OBJECT

public:
    /** @brief Initializes the WebSocket and reconnect timer. */
    explicit Chatclient();

    /** @brief Validates the server address and starts connecting. */
    void connectToServer(const QUrl &url);
    /** @brief Disconnects and stops further connection attempts. */
    void disconnectFromServer();
    /** @brief Sends a text message when connected. */
    void sendMessage(const QString &message);

signals:
    /** @brief Signals a successful connection. */
    void connected();
    /** @brief Provides a description of a connection error. */
    void connectionError(const QString &message);
    /** @brief Signals an explicitly requested disconnection. */
    void closed();
```

```
    /** @brief Provides a received text message. */
    void messageReceived(const QString &message);
    /** @brief Signals a scheduled reconnection attempt. */
    void reconnecting();

public slots:
    /** @brief Accepts a requested connection and stops the timer. */
    void onConnected();
    /** @brief Cancels reconnection and disconnects. */
    void onCancelReconnect();

private slots:
    /** @brief Schedules reconnection after a connection loss. */
    void onDisconnected();
    /** @brief Forwards a received message as a signal. */
    void onTextMessageReceived(const QString &message);
    /** @brief Reports socket errors and schedules reconnection. */
    void onSocketError();
    /** @brief Reconnects to the last address when the timer expires. */
    void onReconnectTimeout();

private:
    /** @brief Starts the reconnect timer if needed. */
    void scheduleReconnect();
    bool connectionRequested = false;
    QWebSocket mSocket;
    QTimer reconnectTimer;
    QUrl lastUrl;
    const std::chrono::milliseconds tReconnectTimout = std::chrono::milliseconds(500);
};

#endif // CHATCLIENT_H
```

Quellcode 5: chatclient.cpp

```
#include "chatclient.h"
#include <QDebug>

Chatclient::Chatclient()
{

    connect(&mSocket, SIGNAL(connected()), this, SLOT(onConnected()));
    connect(&mSocket, SIGNAL(disconnected()), this, SLOT(onDisconnected()));
    connect(&mSocket, SIGNAL(textMessageReceived(QString)), this,
            SLOT(onTextMessageReceived(QString)));
    connect(&mSocket, SIGNAL(errorOccurred(QAbstractSocket::SocketError)), this,
            SLOT(onSocketError()));

    reconnectTimer.setSingleShot(true);
    connect(&reconnectTimer, SIGNAL(timeout()), this, SLOT(onReconnectTimeout()));
    qDebug() << "[Chatclient] Initialized";
}

void Chatclient::connectToServer(const QUrl &url)
{
    // Prevent the previous socket's disconnect signal from scheduling a reconnect.
    reconnectTimer.stop();
    connectionRequested = false;
    mSocket.abort();
    qDebug() << "[Chatclient] Connecting to server:" << url;

    if (!url.isValid() || url.host().isEmpty()) {
        qDebug() << "[Chatclient] Invalid WebSocket address:" << url;
        emit connectionError(tr("Ungültige WebSocket-Adresse"));
```

```
30          return;
31      }
32      lastUrl = url;
33      connectionRequested = true;
34      mSocket.open(url);
35  }
36
37  void Chatclient::disconnectFromServer()
38  {
39      qDebug() << "[Chatclient] Disconnect requested; cancelling automatic reconnect";
40      // An explicit disconnect must also cancel automatic reconnection.
41      connectionRequested = false;
42      reconnectTimer.stop();
43      mSocket.abort();
44      emit closed();
45  }
46
47  void Chatclient::sendMessage(const QString &message)
48  {
49      if (mSocket.state() == QAbstractSocket::ConnectedState) {
50          const qint64 bytesSent = mSocket.sendTextMessage(message);
51          qDebug() << "[Chatclient] Send result:" << bytesSent
52                   << "bytes for" << message.size() << "characters";
53      } else {
54          qDebug() << "[Chatclient] Message not sent; socket state:" << mSocket.state();
55      }
56  }
57
58  void Chatclient::onConnected()
59  {
60      reconnectTimer.stop();
61      qDebug() << "[Chatclient] WebSocket connected; reconnect timer stopped";
62      emit connected();
63  }
64
65  void Chatclient::onSocketError()
66  {
67      qDebug() << "[Chatclient] Socket error:" << mSocket.errorString()
68               << "state:" << mSocket.state();
69      if (!connectionRequested)
70          return;
71      scheduleReconnect();
72      emit connectionError(mSocket.errorString());
73  }
74
75  void Chatclient::onDisconnected()
76  {
77      qDebug() << "[Chatclient] WebSocket disconnected; reconnect requested:"
78               << connectionRequested;
79      if (connectionRequested)
80          scheduleReconnect();
81  }
82
83  void Chatclient::scheduleReconnect()
84  {
85      // Error and disconnect signals can both arrive; schedule only one retry.
86      if (!connectionRequested || reconnectTimer.isActive())
87          return;
88      reconnectTimer.start(tReconnectTimout);
89      qDebug() << "[Chatclient] Reconnect scheduled in" << reconnectTimer.interval() << "ms";
90      emit reconnecting();
91  }
92
```

```
93  void Chatclient::onTextMessageReceived(const QString &message)
94  {
95      qDebug() << "[Chatclient] WebSocket received:" << message.size() << "characters";
96
97      emit messageReceived(message);
98  }
99
100 void Chatclient::onReconnectTimeout()
101 {
102     if (!connectionRequested)
103         return;
104     qDebug() << "[Chatclient] Reconnecting to:" << lastUrl;
105     mSocket.open(lastUrl);
106 }
107
108 void Chatclient::onCancelReconnect()
109 {
110     qDebug() << "[Chatclient] Reconnect cancelled";
111     disconnectFromServer();
112 }
```

A.4. Benutzeroberfläche

Quellcode 6: mainwindow.h

```h
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QLayout>
#include <QDebug>
#include <QLabel>
#include <QLineEdit>
#include <QPlainTextEdit>
#include <QPushButton>
#include "chatclient.h"
#include "emojireplacer.h"

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

/** @brief Chat interface for managing connections and displaying messages. */
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    /** @brief Creates the interface and connects its signals and slots. */
    explicit MainWindow(QWidget *parent = nullptr);
    /** @brief Releases the UI helper object. */
    ~MainWindow() override;

public slots:
    /** @brief Connects to the entered address and clears the chat history. */
    void btnConnectClicked();
    /** @brief Disconnects and updates the interface. */
    void btnDisconnectClicked();
    /** @brief Sends and displays the text with emojis and the username. */
    void btnSendClicked();

private slots:
    /** @brief Updates the interface after a successful connection. */
    void onConnected();
    /** @brief Displays the ongoing reconnection attempt. */
    void onReconnecting();
    /** @brief Sets the interface to the disconnected state. */
    void onClosed();
    /** @brief Displays a connection error as a tooltip. */
    void onConnectionError(const QString &message);
    /** @brief Replaces emoticons and displays the received message. */
    void onMsgRcv(QString msg);

private:
    /** @brief Possible connection states of the interface. */
    enum class ConnectionState { Disconnected, Connected, Error, Reconnecting };
    /** @brief Updates controls and the connection indicator for the given state. */
    void updateConnectionState(ConnectionState state);
    ConnectionState connectionState = ConnectionState::Disconnected;

    Ui::MainWindow *ui;

    QVBoxLayout *mainBoxLayout = nullptr;
```

```
61      QWidget *mainWidget = nullptr;
62
63      QVBoxLayout *topVBoxLayout = nullptr;
64      QHBoxLayout *topConnectionBarLayout = nullptr;
65      QHBoxLayout *topUsernameConfLayout = nullptr;
66      QLabel *serverLabel = nullptr;
67      QLineEdit *serverTextBox = nullptr;
68      QPushButton *connectButton = nullptr;
69      QPushButton *disconnectButton = nullptr;
70      QLabel *userNameLabel = nullptr;
71      QLineEdit *userName = nullptr;
72
73      QPlainTextEdit *chatHistory = nullptr;
74
75      QHBoxLayout *bottomSendLayout = nullptr;
76      QLineEdit *msgToSend = nullptr;
77      QPushButton *sendButton = nullptr;
78
79      Chatclient chatClient;
80  };
81  #endif // MAINWINDOW_H
```

Quellcode 7: mainwindow.cpp

```
1   #include "mainwindow.h"
2   #include "./ui_mainwindow.h"
3   #include <QDebug>
4
5   MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent), ui(new Ui::MainWindow)
6   {
7       ui->setupUi(this);
8
9       mainBoxLayout = new QVBoxLayout;
10      mainWidget = new QWidget;
11      mainWidget->setLayout(mainBoxLayout);
12      setCentralWidget(mainWidget);
13
14      /* Top Server and Connect Bar */
15      topVBoxLayout = new QVBoxLayout;
16      topConnectionBarLayout = new QHBoxLayout;
17      topUsernameConfLayout = new QHBoxLayout;
18
19      serverLabel = new QLabel("Server:");
20      serverTextBox = new QLineEdit();
21      serverTextBox->setText("localhost:1234");
22      connectButton = new QPushButton("verbinden");
23      disconnectButton = new QPushButton("trennen");
24
25      topConnectionBarLayout->addWidget(serverLabel);
26      topConnectionBarLayout->addWidget(serverTextBox);
27      topConnectionBarLayout->addWidget(connectButton);
28      topConnectionBarLayout->addWidget(disconnectButton);
29
30      userNameLabel = new QLabel("Benutzername");
31      userName = new QLineEdit;
32      userName->setText("Simon");
33
34      topUsernameConfLayout->addWidget(userNameLabel);
35      topUsernameConfLayout->addWidget(userName);
36
37      topVBoxLayout->addLayout(topConnectionBarLayout);
38      topVBoxLayout->addLayout(topUsernameConfLayout);
39
40      mainBoxLayout->addLayout(topVBoxLayout);
```

```
41
42      /* Chat Box */
43      chatHistory = new QPlainTextEdit;
44      chatHistory->setReadOnly(true);
45      mainBoxLayout->addWidget(chatHistory);
46
47      /* Message input and send button */
48      bottomSendLayout = new QHBoxLayout;
49      msgToSend = new QLineEdit;
50      msgToSend->setReadOnly(false);
51      sendButton = new QPushButton("senden");
52
53      bottomSendLayout->addWidget(msgToSend);
54      bottomSendLayout->addWidget(sendButton);
55
56      mainBoxLayout->addLayout(bottomSendLayout);
57
58      /* Connect signal and slots */
59      connect(connectButton, SIGNAL(clicked()), this, SLOT(btnConnectClicked()));
60      connect(disconnectButton, SIGNAL(clicked()), this, SLOT(btnDisconnectClicked()));
61      connect(sendButton, SIGNAL(clicked()), this, SLOT(btnSendClicked()));
62      connect(msgToSend, SIGNAL(returnPressed()), this, SLOT(btnSendClicked()));
63
64      connect(&chatClient, SIGNAL(messageReceived(QString)), this, SLOT(onMsgRcv(QString)));
65
66      connect(&chatClient, SIGNAL(connected()), this, SLOT(onConnected()));
67      connect(&chatClient, SIGNAL(closed()), this, SLOT(onClosed()));
68      connect(&chatClient, SIGNAL(connectionError(QString)), this, SLOT(onConnectionError(
        QString)));
69
70      connect(&chatClient, SIGNAL(reconnecting()), this, SLOT(onReconnecting()));
71
72      updateConnectionState(ConnectionState::Disconnected);
73  }
74
75  void MainWindow::onConnected()
76  {
77      updateConnectionState(ConnectionState::Connected);
78  }
79
80  void MainWindow::onClosed()
81  {
82      updateConnectionState(ConnectionState::Disconnected);
83  }
84
85  void MainWindow::onReconnecting()
86  {
87      updateConnectionState(ConnectionState::Reconnecting);
88  }
89
90  void MainWindow::onConnectionError(const QString &message)
91  {
92      qDebug() << "[MainWindow] Connection error:" << message;
93      // Keep the reconnect indicator visible while automatic retries are active.
94      if (connectionState != ConnectionState::Reconnecting)
95          updateConnectionState(ConnectionState::Error);
96  }
97
98  void MainWindow::btnConnectClicked()
99  {
100     qDebug() << "[MainWindow] Connect button clicked";
101     QString address = serverTextBox->text();
102     address.prepend("ws://");
```

```
103
104     chatClient.connectToServer(QUrl(address));
105     chatHistory->clear();
106 }
107
108 void MainWindow::btnDisconnectClicked()
109 {
110     qDebug() << "[MainWindow] Disconnect button clicked";
111     updateConnectionState(ConnectionState::Disconnected);
112     chatClient.disconnectFromServer();
113 }
114
115 void MainWindow::updateConnectionState(ConnectionState state)
116 {
117     // update private member
118     connectionState = state;
119     const bool connected = state == ConnectionState::Connected;
120     // Allow cancellation during reconnects, but keep the server address locked.
121     const bool active = connected || state == ConnectionState::Reconnecting;
122
123     userName->setEnabled(connected);
124     userNameLabel->setEnabled(connected);
125     msgToSend->setEnabled(connected);
126     sendButton->setEnabled(connected);
127     chatHistory->setEnabled(connected);
128     disconnectButton->setEnabled(active);
129     serverTextBox->setEnabled(!active);
130     connectButton->setEnabled(!active);
131
132     if (connected) {
133         qDebug() << "[MainWindow] Connection state: connected";
134         connectButton->setText(tr("verbunden"));
135         connectButton->setStyleSheet("background-color: #238636; color: white;");
136     } else if (state == ConnectionState::Error) {
137         qDebug() << "[MainWindow] Connection state: error";
138         connectButton->setText(tr("erneut verbinden"));
139         connectButton->setStyleSheet("background-color: #c62828; color: white;");
140     } else if (state == ConnectionState::Reconnecting) {
141         qDebug() << "[MainWindow] Connection state: reconnecting";
142         connectButton->setText(tr("reconnect"));
143         connectButton->setStyleSheet("background-color: #e4e120; color: black;");
144         connectButton->setEnabled(false);
145     } else {
146         qDebug() << "[MainWindow] Connection state: disconnected";
147         connectButton->setText("verbinden");
148         connectButton->setStyleSheet(QString());
149     }
150 }
151
152 void MainWindow::btnSendClicked()
153 {
154     qDebug() << "[MainWindow] Sending message with" << msgToSend->text().size() << "
    characters";
155     QString replacedMsg(msgToSend->text());
156
157     // Convert only the message body so emoticons in the username stay unchanged.
158     EmojiReplacer::replaceToEmojis(replacedMsg);
159
160     QString msg(userName->text() + ": " + replacedMsg);
161     chatClient.sendMessage(msg);
162     // Show the outgoing message immediately without waiting for a server echo.
163     chatHistory->appendPlainText(msg);
164     msgToSend->clear();
```

```
165 }
166
167 void MainWindow::onMsgRcv(QString msg)
168 {
169     qDebug() << "[MainWindow] Displaying received message with" << msg.size() << "characters
    ";
170     EmojiReplacer::replaceToEmojis(msg);
171     chatHistory->appendPlainText(msg);
172 }
173
174 MainWindow::~MainWindow()
175 {
176     delete ui;
177 }
```

Quellcode 8: mainwindow.ui

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <ui version="4.0">
3   <class>MainWindow</class>
4   <widget class="QMainWindow" name="MainWindow">
5    <property name="geometry">
6     <rect>
7      <x>0</x>
8      <y>0</y>
9      <width>800</width>
10     <height>600</height>
11    </rect>
12   </property>
13   <property name="windowTitle">
14    <string>MainWindow</string>
15   </property>
16   <widget class="QWidget" name="centralwidget"/>
17   <widget class="QMenuBar" name="menubar">
18    <property name="geometry">
19     <rect>
20      <x>0</x>
21      <y>0</y>
22      <width>800</width>
23      <height>18</height>
24     </rect>
25    </property>
26   </widget>
27   <widget class="QStatusBar" name="statusbar"/>
28  </widget>
29  <resources/>
30  <connections/>
31 </ui>
```

A.5. Emoji-Ersetzung

Quellcode 9: emojireplacer.h

```h
#ifndef EMOJIREPLACER_H
#define EMOJIREPLACER_H

#include <qmap.h>
#include <qstring.h>

/** @brief Replaces supported text emoticons with Unicode emojis. */
class EmojiReplacer
{
public:
    /** @brief Creates an emoji replacer. */
    EmojiReplacer();

    /** @brief Replaces emoticons in the given text in place.
     *  @param[in,out] inplaceString Text to modify.
     */
    static void replaceToEmojis(QString &inplaceString);

private:
    inline static const QMap<QString, QString> replacements{{":)", QString::fromUtf8("
    [U+1F642]")},
                                                {":(", QString::fromUtf8("[U+1F641]")},
                                                {";)", QString::fromUtf8("[U+1F609]")},
                                                {":D", QString::fromUtf8("[U+1F603]")}};
};

#endif // EMOJIREPLACER_H
```

Quellcode 10: emojireplacer.cpp

```cpp
#include "emojireplacer.h"

EmojiReplacer::EmojiReplacer() {}

void EmojiReplacer::replaceToEmojis(QString &inplaceString)
{
    for (auto it = replacements.cbegin(); it != replacements.cend(); ++it) {
        inplaceString.replace(it.key(), it.value());
    }
}
```